

Algoritmi de tipul Ciurul lui Eratostene

1. Ideea generala

Ciurul clasic marcheaza multiplii pentru a obtine toate numerele prime pana la n . Prin 'algoritmi de tip ciur' intelegem tehnici similare: parcurgem i si actualizam multiplii lui i pentru a calcula rapid o informatie pentru toate numerele pana la n (prime, cel mai mic divizor, numar de divizori, suma divizorilor, ϕ , etc.).

2. Tipar comun (schema)

Avem un vector $v[0..n]$ initializat (de obicei cu 0/true). Pentru fiecare i , daca i 'indeplineste conditia' (ex: este prim sau are $v[i] \neq 0$), atunci actualizam toate valorile de forma $j = i, 2i, 3i, \dots$ (multiplii).

Exemplu de schema (pseudocod):

```
for i = 2..n:
    if i este 'baza' (ex: prim):
        for j = i*i, i*i+i, ... <= n:
            marcheaza/actualizeaza j
```

3. Ciur clasic (reper)

Ciurul clasic produce un vector `prim[]` care spune daca fiecare numar este prim. Este baza pentru celelalte variante.

4. Ciur pentru cel mai mic divizor (SPF = smallest prime factor)

In loc sa marcam doar 'neprim', putem retine pentru fiecare numar compus cel mai mic divizor prim. Asta permite descompunerea rapida in factori primi pentru multe numere (fara sa refaci sqrt pentru fiecare).

Exemplu explicativ ($n=20$):

$\text{spf}[2]=2, \text{spf}[3]=3, \text{spf}[4]=2, \text{spf}[5]=5, \text{spf}[6]=2, \text{spf}[7]=7, \text{spf}[8]=2, \text{spf}[9]=3, \text{spf}[10]=2, \dots$
Descompunere 18: $\text{spf}[18]=2 \rightarrow 18=2*9, \text{spf}[9]=3 \rightarrow 9=3*3 \Rightarrow 18 = 2 * 3^2$.

Implementare minimala SPF (fara STL)

```
#include <iostream>
using namespace std;

int main(){
    int n; cin >> n;
    static int spf[1000001];
    for(int i=0; i<=n; i++) spf[i]=0;
    for(int i=2; i<=n; i++){
        if(spf[i]==0){
            spf[i]=i; // i prim
            if(1LL*i*i<=n)
                for(int j=i*i; j<=n; j+=i)
                    if(spf[j]==0) spf[j]=i;
        }
    }
    for(int i=2; i<=n && i<=20; i++) cout<<i<<": "<<spf[i]<<" ";
    return 0;
}
```

```
}
```

Cum folosesti spf ca sa descompui un numar x (idee):

Cat timp $x > 1$: $p = \text{spf}[x]$, numaram de cate ori se imparte la p . Asta e mult mai rapid decat sa testezi divizori pana la $\sqrt{x}$, mai ales cand ai multe intrebari.

```
while(x > 1){  
    int p = spf[x];  
    int cnt = 0;  
    while(x % p == 0){ x/=p; cnt++; }  
    // avem  $p^{\text{cnt}}$   
}
```

5. Ciur pentru numarul de divizori

Vrem $d[k]$ = numarul de divizori ai lui k . Fiecare i este divizor pentru multiplii sai ($i, 2i, 3i, \dots$). Deci pentru fiecare i , crestem $d[j]$ pentru toti multiplii j ai lui i .

Exemplu ($n=10$):

Pornim cu $d[] = 0$.

$i=1 \rightarrow$ crestem $d[1..10]$ (toate au divizor 1)

$i=2 \rightarrow$ crestem $d[2], d[4], d[6], d[8], d[10]$

$i=3 \rightarrow$ crestem $d[3], d[6], d[9]$

...

La final: $d[6]=4$ (divizori: 1,2,3,6), $d[10]=4$ (1,2,5,10).

Implementare minimala (numar de divizori pana la n)

```
#include <iostream>
using namespace std;

int main(){
    int n; cin >> n;
    static int d[1000001];
    for(int i=1; i<=n; i++) d[i]=0;
    for(int i=1; i<=n; i++)
        for(int j=i; j<=n; j+=i)
            d[j]++;
    for(int i=1; i<=n && i<=10; i++) cout<<i<<": "<<d[i]<<" ";
    return 0;
}
```

6. Ciur pentru suma divizorilor

Analog, putem calcula $s[k]$ = suma divizorilor lui k . Pentru fiecare i , il adunam la $s[j]$ pentru toti multiplii j ai lui i .

Exemplu ($n=8$):

Suma divizorilor lui 8 este $1+2+4+8 = 15$. Ciurul va aduna 1 la toate, 2 la multiplii lui 2, 4 la multiplii lui 4, 8 la multiplii lui 8.

Implementare minimala (suma divizorilor)

```
for(int i=1; i<=n; i++)
    for(int j=i; j<=n; j+=i)
        s[j] += i;
```

7. Ciur pentru Euler phi (functia totient)

$\phi(x)$ = numarul de valori k cu $1 \leq k \leq x$ si $\gcd(k, x) = 1$. Se poate calcula pentru toate valorile pana la n printr-un ciur: initial $\phi[i]=i$, iar pentru fiecare prim p , pentru toti multiplii j ai lui p aplicam $\phi[j] = \phi[j] / p * (p-1)$.

Exemplu (idee):

Pentru $x=12$, factorii primi sunt 2 si 3.

$\phi(12) = 12 * (1 - 1/2) * (1 - 1/3) = 12 * (1/2) * (2/3) = 4$.

Implementare minimala phi (fara STL)

```
#include <iostream>
using namespace std;

int main(){
    int n; cin >> n;
    static int phi[1000001];
    for(int i=0; i<=n; i++) phi[i]=i;
    for(int i=2; i<=n; i++){
        if(phi[i]==i){ // i prim
            for(int j=i; j<=n; j+=i)
                phi[j] = phi[j]/i*(i-1);
        }
    }
    for(int i=1; i<=n && i<=10; i++) cout<<i<<": "<<phi[i]<<" ";
    return 0;
}
```

8. Ciur 'de acoperire' (marcari de multipli)

Uneori nu ne intereseaza primele, ci doar sa marcam multiplii pentru o proprietate.

Exemple:

- marcarea numerelor care sunt multiplii unor valori (ex: multipli de 4 sau 6);
- precomputari de tip `areFactor[i]` sau `viz[i]` pentru a raspunde rapid la intrebari.

Exemplu simplu:

Vrem sa marcam toate numerele ≤ 30 care sunt multipli de 4 sau 6.

Marcam: 4,8,12,16,20,24,28 si 6,12,18,24,30. Observam ca 12 si 24 apar in ambele liste.

9. Cand folosim (alegere rapida)

- O singura verificare de primalitate: test $O(\sqrt{n})$.
- Multe verificari pana la n : ciur clasic.
- Multe descompuneri in factori primi: ciur SPF.
- Numar/suma divizori pentru toate valorile: ciur pe divizori.
- phi pentru toate valorile: ciur phi.

10. Complexitati (orientativ)

- ciur clasic prime: $O(n \log \log n)$, memorie $O(n)$
- ciur SPF: timp similar, memorie $O(n)$
- ciur numar divizori / suma divizori: $O(n \log n)$, memorie $O(n)$
- ciur phi: foarte eficient in practica, memorie $O(n)$

11. Greseli frecvente

- dimensiune insuficienta a vectorilor (n prea mare fata de alocare)
- neinitializarea corecta (ex: `prim[0]=prim[1]=false`)
- inceperea marcarii multiplilor de la $2*i$ in loc de $i*i$ (la prime)
- uitarea unei conditii de tip $r+1 < n$ cand se lucreaza cu intervale (ex: difference array)
- folosirea tipului `int` cand sumele pot depasi `int` (foloseste `long long`)

12. Checklist de BAC

- stii schema 'parcureaza baza i si actualizeaza multiplii j '
- stii cel putin doua aplicatii: prime si SPF/divizori
- poti explica un exemplu mic cap-coada
- stii cum se refolosesc precomputarile pentru raspuns rapid